Contents

Parametric polymorphism provides a language-independent abstraction, its implementation varies considerably among programming languages and compiler architectures.

The principal design choice concerns how generic types are represented after compilation and when concrete type instantiations are produced.

These implementation strategies influence several aspects of the execution model, including runtime representation, reflection capabilities, performance, and binary size.

The following dimensions constitute the primary axes along which implementations of parametric polymorphism are typically compared.

0.1 Instantiation Time

A first distinguishing characteristic is the stage at which generic definitions are instantiated.

Some implementations perform instantiation entirely during compilation.

In this approach, each concrete use of a generic function or data structure is expanded into a specialized implementation before executable code is generated.

Since all specializations already exist in the final binary, runtime execution incurs no additional generic instantiation cost.

Other implementations compile generic definitions only once.

Generic type parameters are verified during type checking and subsequently removed or replaced by a common representation.

Consequently, no additional executable code is generated for different type arguments, and all instantiations share the same compiled implementation.

0.2 Runtime Representation

Implementations also differ in how generic types are represented after compilation.

Type erasure removes generic type parameters, producing a homogeneous representation where different instantiations share the same executable code.

The runtime system therefore treats different generic instantiations as equivalent representations.

Conversely, *reified generics* preserve concrete type information by generating distinct specialized code for each instantiated type.

Each generic instantiation may therefore possess its own runtime representation, memory layout, and executable instructions.

0.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the chosen representation strategy.

When type erasure is employed, generic parameters are generally unavailable after compilation.

Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation.

Runtime reflection or runtime type identification (RTTI) may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis.

0.4 Type Safety

Both implementation strategies preserve static type safety, although they achieve this objective differently.

In implementations based on type erasure, generic correctness is verified entirely during compilation.

Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system.

Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly.

Since each specialization corresponds to a concrete type, correctness is preserved without requiring erased representations or implicit runtime casts.

0.5 Performance

The implementation strategy significantly influences both compilation and execution performance.

Homogeneous implementations generally produce smaller binaries and reduce compilation time because generic definitions are compiled only once.

However, the shared representation may require additional indirections, boxing, or runtime casts, limiting optimization opportunities.

Heterogeneous implementations incur greater compilation cost and larger executables because every instantiated type generates a distinct implementation.

Nevertheless, the compiler can aggressively optimize each specialization, exploiting concrete type information for improved instruction selection, memory layout, inlining, and register allocation.

Consequently, runtime performance often approaches that of manually specialized code.

0.6 Summary

The implementation of parametric polymorphism is primarily characterized by five orthogonal dimensions:

1. the stage at which generic instantiation occurs;
2. the runtime representation of generic types;

3. the availability of runtime type information and reflection;

4. the trade-off between compilation cost, binary size, and execution performance;

5. the mechanisms through which static type safety is preserved.

These dimensions form the basis for comparing homogeneous implementations based on type erasure with heterogeneous implementations based on reification or monomorphization.

Chapter 1

C++ Templates

1.1 Instantiation Time

C++ implements parametric polymorphism through the *template* mechanism, which adopts a purely compile-time instantiation model.

Unlike languages that compile a generic definition only once, a C++ template is not itself an executable entity. Instead, it represents a parameterized program definition that serves as a blueprint from which the compiler generates independent concrete implementations.

From the perspective of instantiation time, the defining characteristic of C++ is that executable code is generated only after the compiler knows the complete set of template arguments.

Consequently, all template instantiations are resolved entirely during static compilation, and no template instantiation mechanism exists during program execution.

The complete instantiation process can be understood by following the template through the compilation pipeline.

1.1.1 Template Definition

When the compiler encounters a template definition, it first performs the ordinary front-end compilation stages, including lexical analysis, parsing, and semantic analysis that can be performed without knowledge of future template arguments.

For example:

```
template<typename T>
T get_max(T a, T b)
{
    return (a > b) ? a : b;
}
```

The compiler parses the template syntax, builds the corresponding Abstract Syntax Tree (AST), checks syntactic correctness, and stores an internal parameterized representation.

At this stage, the compiler does not generate executable machine code.

The reason is straightforward: the compiler still ignores the concrete meaning of the template parameter `T`, and therefore cannot determine the exact operations, object layout, calling convention, or generated instructions required by the final implementation.

Consequently, the template definition remains a generic blueprint waiting for future instantiation.

1.1.2 Instantiation Trigger

The instantiation process begins only when the compiler encounters a concrete use of the template, either through an explicit specification of the template argument or through implicit type deduction.

Example.

Consider the following template invocations:

```
int i = get_max(3, 7);
double d = get_max<double>(2.5, 4.8);
```

The first invocation relies on template argument deduction:

$$T = int$$

whereas the second invocation explicitly specifies the template argument:

$$T = double$$

Therefore, the compiler generates two different template instantiations: `get_max<int>` and `get_max<double>`

Each unique set of template arguments constitutes a distinct template instantiation. Instantiation may occur through:

- `implicit instantiation`, automatically performed whenever a specialization is required by the program;
- `explicit instantiation`, requested directly by the programmer through an explicit instantiation declaration or definition;
- `explicit specialization`, where an entirely different implementation is provided for a specific combination of template arguments.

Regardless of how instantiation is initiated, executable code generation begins only after the complete set of template arguments is known.

1.1.3 Template Argument Substitution

Once instantiation has been triggered, the compiler performs template argument substitution.

During this phase, every formal template parameter is replaced by its corresponding concrete argument.

From previous example.

$$T \to int$$

produces the specialization:

```
int get_max(int a, int b)
{
    return (a > b) ? a : b;
}
```

whereas:

$$T \to double$$

produces:

```
double get_max<double>(double a, double b)
{
    return (a > b) ? a : b;
}
```

Conceptually, the compiler now treats each specialization as if it had been written explicitly by the programmer.

From this point onward, the specialization behaves as an ordinary non-template function.

1.1.4 Semantic Analysis of the Specialization

Instantiation does not immediately produce executable code.

Instead, each generated specialization undergoes the normal semantic analysis performed for ordinary C++ program entities.

Depending on the specialization, the compiler performs activities such as: type checking, overload resolution, name lookup, constraint verification for constrained templates, constant expression evaluation, instantiation of nested templates and verification of language rules required by the selected template arguments.

This step is fundamental because many semantic properties cannot be verified before template parameters have been substituted.

Consequently, a template definition that is syntactically correct may still produce compilation errors if one of its concrete instantiations violates the requirements imposed by the template body.

1.1.5 Intermediate Representation and Code Generation

Once semantic analysis has successfully completed, the specialization enters the compiler back-end.

The compiler generates an Intermediate Representation (IR) specifically for the instantiated specialization.

Subsequently, the specialization undergoes the standard optimization pipeline, which may include: constant propagation, dead-code elimination, function inlining, loop optimizations, register allocation, instruction selection and target-specific optimizations.

Finally, the compiler emits native machine code corresponding exclusively to that specialization.

Importantly, this process is repeated independently for every distinct template instantiation.

Consequently, each specialization owns an independent executable implementation. Infact, the compiler does not produce a single shared executable representation capable of serving multiple template arguments.

1.1.6 Completion of the Instantiation Process

The instantiation process terminates entirely before linking.

By the time object files are generated, every template specialization required by the translation unit already exists as ordinary compiled code.

After linking, the final executable therefore contains every specialization required by the program.

From the perspective of executable code generation, the lifecycle of a C++ template can be summarized as follows:

1. The compiler parses the template definition.
2. The template is stored as a parameterized internal representation.
3. A concrete template use triggers instantiation.
4. Template parameters are substituted with concrete arguments.
5. The generated specialization undergoes complete semantic analysis.
6. The compiler produces a specialization-specific intermediate representation.
7. Optimization passes are applied.
8. Native machine code is emitted.
9. The specialization is included in the final executable before program execution begins.

Therefore, the defining characteristic of C++ parametric polymorphism is that the entire instantiation process is completed during static compilation.

When execution begins, no template instantiation mechanism remains active.

The runtime simply executes machine code that has already been generated for every required specialization during compilation.

1.2 Runtime Representation

C++ templates are implemented according to a translation model in which generic source-level abstractions are transformed into concrete program entities before execution. A template declaration itself does not correspond to a runtime object, runtime function, or executable structure. Instead, it defines a family of possible entities that are materialized only when the compiler generates a specific specialization.

The runtime representation of a C++ template specialization is therefore equivalent to the representation of an ordinary C++ function, class, or object. After compilation and linking, the executable contains native machine instructions and data structures associated with each instantiated specialization. The execution environment, including the operating system loader and the processor, has no knowledge of the original template declaration, template parameters, or instantiation process. It only observes ordinary executable artifacts such as functions, symbols, and memory objects.

A template can be conceptually modeled as a compile-time transformation:

$$T : (A_1, A_2, ..., A_n) \rightarrow P$$

where T represents a template definition, $(A_1, A_2, ..., A_n)$ are template arguments, and P is a concrete program entity generated from the template.

For example:

$$T < int > \rightarrow P_{int}$$

$$T < double > \rightarrow P_{double}$$

The generated entities are independent concrete implementations. During execution, the program does not invoke a generic representation of T; instead, it executes the machine code generated for each concrete specialization.

1.2.1 Runtime Form of Template Specializations

The C++ compilation model transforms template specializations into ordinary binary-level entities. Consider the following function template:

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

When the program requires multiple specializations:

```
add<int>(1,2);
add<double>(1.0,2.0);
```

the compiler produces separate functions:

$$add < int > (int, int)$$

and

$$add < double > (double, double)$$

At the binary level, these functions are represented as independent machine code regions. Each specialization has its own instructions, symbol identity, and executable address. The runtime system does not contain a generic function body with a runtime type parameter. Instead, it contains the concrete implementations produced during translation.

Once the executable is loaded into memory, the operating system maps the binary sections containing executable instructions and data. Control flow transfers directly to the generated machine-code addresses. The processor executes instructions without any awareness that those instructions were originally produced from a template definition.

Therefore, the runtime representation of:

$$Vector < int >$$

and:

$$Vector < double >$$

is not a single generic runtime object:

$$Vector < T >$$

but two independent concrete program entities:

$$Vector < int >$$

and:

$$Vector < double >$$

Each specialization exists as a normal C++ type produced before execution.

1.2.2 Monomorphization and Native Code Artifacts

C++ uses heterogeneous translation, commonly described as monomorphization, to implement templates. This mechanism transforms generic template definitions into concrete specialized entities for each set of template arguments used by the program.

The essential property of monomorphization is that every specialization receives its own native representation. The compiler substitutes the template arguments into the template definition and generates a concrete function or class implementation.

For example:

```
template <typename T>
struct Container
{
    T value;
};

Container<int> a;
Container<float> b;
```

creates two independent specializations:

$$Container < int >$$

and

$$Container < float >$$

The executable contains the concrete representations of these entities. There is no runtime object representing:

$$Container < T >$$

because the generic abstraction has already been resolved during translation.

1.2.3 Native Machine Code Generation

For function templates, each specialization produces a distinct machine-code artifact.

Conceptually:

$$Compiler(T < int >) = MachineCode_{int}$$

$$Compiler(T < double >) = MachineCode_{double}$$

These generated artifacts behave exactly like functions written explicitly for those types.

For example, after compilation, the following two functions:

$$sort < int > ()$$

and:

$$sort < double > ()$$

are ordinary executable functions. They have independent entry points and can be invoked through normal machine-level control-flow instructions.

The binary format does not preserve the template abstraction. The executable contains:

- machine instructions for instantiated functions;

- symbols identifying generated entities;
- relocation information required during linking;
- optional runtime metadata produced by other C++ mechanisms.

These artifacts are not different from those produced by non-template C++ code.

1.2.4 Final Executable Representation

After linking, the final executable represents a collection of concrete compiled entities:

$$Executable = \{Code(P_1), Code(P_2), ..., Code(P_n)\}$$

where each P_i corresponds to a particular template specialization.

During execution, the processor does not interpret template parameters. A machine instruction such as a function call transfers execution to a specific address. The target address corresponds to a compiled function, not to a generic template definition.

The relationship between a generated function and its original template source exists only within compiler-generated information such as debugging information or symbol naming conventions. It is not part of the runtime execution model.

1.2.5 Absence of Generic Metadata and the Role of RTTI

A defining characteristic of the C++ template implementation model is the absence of runtime generic metadata. After template instantiation has occurred, the executable does not contain information describing the original template declaration or the possible template arguments.

The runtime environment does not know that a particular function originated from:

```
template<typename T>
void process(T value);
```

Instead, it only observes a concrete function generated from a specific instantiation:

$$process < int > ()$$

or:

$$process < double > ()$$

The following concepts do not exist as runtime entities:

- template declarations;
- template parameter lists;

- generic type variables;
- mechanisms for creating new template specializations dynamically.

The execution environment only manipulates the final native representation.

1.2.6 Name Mangling

Compilers often encode template arguments into symbol names through name mangling. This allows different specializations to coexist in object files and allows the linker to distinguish between entities such as:

$$function < int > ()$$

and:

$$function < double > ()$$

However, name mangling is not runtime metadata. It is a compiler and linker mechanism used during binary generation. Once linking is completed, the CPU does not interpret mangled names or template information.

The executable address associated with a symbol identifies machine code. The processor executes this code without understanding the source-level template relationship.

1.2.7 Run-Time Type Information

C++ provides optional Run-Time Type Information (RTTI), mainly through features such as:

```
typeid
```

and:

```
dynamic_cast
```

RTTI allows certain concrete C++ types to be identified during execution. However, RTTI does not represent templates themselves.

For example:

```
template<typename T>
class Base
{
public:
    virtual ~Base(){}
};

Base<int> object;
```

may generate runtime type information describing:

$$Base < int >$$

but it does not generate a runtime descriptor describing:

$$Base < T >$$

because the generic template abstraction has no runtime existence.

RTTI therefore represents concrete runtime types produced by the language implementation, not generic template definitions.

The C++ runtime model is consequently based on fully instantiated entities. Templates influence the generation of executable artifacts, but after translation and loading, execution operates exclusively on concrete native code and concrete data representations.

1.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the representation strategy adopted by the programming language implementation. In systems based on type erasure, generic parameters are removed or abstracted during compilation, preventing direct access to the original concrete type arguments during execution. Consequently, runtime inspection mechanisms can only operate on the residual type information preserved after erasure. Reflection facilities applied to erased generic structures therefore provide limited information about the original instantiations, since the relationship between the generic abstraction and its concrete type parameters is no longer explicitly represented.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation and execution. In such systems, runtime reflection or runtime type identification mechanisms can access metadata associated with the actual instantiated generic types, enabling operations such as generic type inspection, runtime metadata analysis, and specialized processing based on concrete type information.

C++ templates follow a different model from type-erased generic systems because template instantiation is performed through compile-time specialization. Template parameters are not represented as a single runtime generic entity; instead, the compiler generates concrete versions of templates for each required set of template arguments. This process, commonly referred to as *template instantiation* or *monomorphization*, preserves the identity of the concrete types used during compilation. Therefore, C++ templates are based on compile-time type preservation rather than runtime reification.

1.3.1 Template Instantiation and Compile-Time Type Information

A C++ template represents a parameterized definition from which the compiler can generate concrete entities after template arguments become known. For example, a

class template such as:

```
template <typename T>
class Container {
 public:
 T value;
};
```

does not directly correspond to a single runtime type representing all possible values of `T`. Instead, when the template is instantiated with a specific type, the compiler produces a distinct specialization:

```
Container<int>
Container<double>
```

Each specialization has an independent compile-time type identity. The compiler therefore retains complete knowledge of the substituted template arguments during parsing, semantic analysis, overload resolution, and template metaprogramming operations.

This compile-time availability of type information is fundamental to several C++ mechanisms, including template specialization, type traits, and compile-time conditional logic. Facilities such as `std::is_same`, `std::is_class`, and `std::enable_if` operate entirely during compilation and exploit the fact that template arguments are explicitly represented in the instantiated type system.

However, compile-time preservation should not be confused with runtime reflection. After compilation, the C++ execution environment does not generally provide a universal mechanism capable of enumerating arbitrary template parameters, inspecting source-level declarations, or dynamically querying all properties of instantiated template entities. The concrete type information preserved by templates is primarily used by the compiler during code generation.

1.3.2 Runtime Type Identification for Template Instantiations

Although C++ does not provide a complete general-purpose reflection system for templates, the language includes Runtime Type Information (RTTI) mechanisms that allow limited runtime identification of polymorphic objects and concrete types. The primary standard facilities are provided by the operators `typeid` and `dynamic_cast`, together with the `std::type_info` class.

The `typeid` operator obtains runtime type information associated with an expression or a type. When applied to an instantiated template class, it can identify the concrete specialization generated by the compiler. For example:

```
template <typename T>
class Wrapper {
};
```

```
Wrapper<int> a;
Wrapper<double> b;

std::cout << typeid(a).name();
std::cout << typeid(b).name();
```

The two objects correspond to different C++ types, namely `Wrapper<int>` and `Wrapper<double>`. Consequently, the RTTI system treats them as distinct runtime types whenever RTTI metadata is generated for these entities. The resulting `std::type_info` objects represent the runtime identity of the instantiated classes.

The information exposed by `std::type_info` is intentionally limited. It allows comparison of runtime type identities through operations such as equality comparison and provides an implementation-defined textual representation through `name()`. It does not provide access to template arguments, class members, inheritance hierarchies, or source-level metadata in the manner of a full reflection system.

1.3.3 RTTI and Polymorphic Template Classes

The `dynamic_cast` operator provides another RTTI mechanism for performing safe runtime conversions within polymorphic class hierarchies. A class must contain at least one virtual function for RTTI-based dynamic identification to be available.

Template classes can participate in such hierarchies like ordinary C++ classes. For example:

```
class Base {
 public:
 virtual ~Base() = default;
};

template <typename T>
class Derived : public Base {
};

Base* ptr = new Derived<int>();

Derived<int>* result = dynamic_cast<Derived<int>*>(ptr);
```

During execution, `dynamic_cast` uses runtime type information associated with the object to verify whether the actual object type corresponds to the requested target type. In this example, the cast succeeds because the dynamically stored object is an instance of the concrete template specialization `Derived<int>`.

The same operation would fail for an object instantiated with a different template argument, such as `Derived<double>`, because each template specialization represents a distinct C++ type. Therefore, RTTI preserves the identity of instantiated template classes when those classes are part of a polymorphic hierarchy.

1.3.4 Relationship Between Template Reification and RTTI

C++ templates demonstrate a hybrid relationship between type preservation and runtime identification. The template mechanism preserves concrete type information during compilation through specialization generation, but this information is not automatically exposed through a general runtime reflection interface.

The compiler knows the complete instantiated type during template compilation. It can therefore perform static analysis, select specialized implementations, and generate code based on the concrete template arguments. This information belongs to the compile-time type system and is consumed before program execution.

RTTI, in contrast, represents a restricted runtime mechanism. It operates only on the subset of type information explicitly retained by the C++ object model, primarily for polymorphic objects and explicit type identification through `typeid`. Consequently, RTTI can distinguish between instantiated template types when the compiler emits the corresponding runtime metadata, but it cannot reconstruct the complete template definition or inspect arbitrary template parameters dynamically.

Therefore, C++ templates preserve generic type information through compile-time monomorphization rather than through runtime reification. Standard RTTI mechanisms provide runtime access to the identity of concrete instantiated types, but they do not constitute a complete reflection system capable of exposing the full structure of template-generated types.

Chapter 2

Java Generics

2.1 Instantiation Time

Java Generics adopt a compilation strategy based on compile-time instantiation followed by type erasure. During compilation, the Java compiler associates concrete type arguments with generic declarations in order to perform static type checking. However, this association does not lead to the generation of a new specialized class representation.

This design differs fundamentally from the approach adopted by C++ templates. While Java uses instantiation only as a compile-time type verification mechanism, C++ performs template instantiation through monomorphization, generating a separate specialization for every concrete type argument

2.1.1 The Java Generic Instantiation Model

Consider the following generic declaration:

```
class Box<T> {
    private T value;

    public T get() {
        return value;
    }
}
```

When the compiler encounters:

```
Box<String> box = new Box<String>();
```

the type parameter T is instantiated with the argument `String` for the purpose of compile-time verification. The compiler therefore checks that every operation involving `box` respects the constraint:

$$T = String$$

However, this instantiation does not generate a new runtime class:

```
Box<String>
```

Instead, Java produces a single generic class representation that is shared by all possible instantiations.

This behavior contrasts with C++ templates, where:

```
Box<std::string>
Box<int>
```

represent distinct compile-time entities. The C++ compiler generates separate specialized implementations containing the concrete type information required for native code generation.

2.1.2 Type Erasure Pipeline

After generic type checking, the Java compiler applies type erasure. The type parameter is removed and replaced by its erasure according to the rules defined by the Java Language Specification.

For an unbounded parameter:

$$< T >$$

the erased type becomes:

$$Object$$

Therefore:

```
class Box<T>
```

is transformed into a representation equivalent to:

```
class Box {
    Object value;

    Object get()
}
```

At this point, Java has completed the generic instantiation process. No additional class corresponding to:

```
Box<String>
```

exists in the JVM.

This differs from C++ template compilation, where the instantiation phase does not remove the concrete type information but instead materializes it into a specialized implementation.

2.1.3 Instantiation with Type Bounds

When a Java generic parameter has a bound:

```
class NumericBox<T extends Number>
```

the compiler verifies that every instantiation satisfies:

$$T <: Number$$

The bound also determines the erasure type. According to the Java Language Specification, the erasure of a type variable corresponds to the erasure of its leftmost bound

Therefore:

```
NumericBox<Integer>
NumericBox<Double>
```

are both translated into a representation equivalent to:

```
NumericBox
```

with:

```
Number value;
```

Conversely, a C++ template:

```
NumericBox<int>
NumericBox<double>
```

would generate two independent specializations with different internal representations

2.1.4 Compiler Adaptation After Erasure

Because Java removes generic parameters before execution, the compiler must insert additional bytecode instructions to preserve the semantics established during compile-time instantiation.

For example:

```
String s = box.get();
```

is compiled into a call returning:

```
Object get()
```

followed by an inserted cast:

```
checkcast java/lang/String
```

C++ templates do not require this mechanism because the generated function already contains the concrete instantiated type. The specialization itself encodes the required type information at compile time

2.2 Runtime Representation

Java Generics are implemented through a *type erasure* model, in which generic type parameters are removed from the executable representation before runtime. The generic abstraction exists only during compilation, where the compiler performs static type checking, verifies type constraints, and generates bytecode compatible with the non-generic JVM execution model.

The fundamental design principle of Java Generics is that different parameterizations of the same generic class share a single runtime representation. For example, the source-level types

$$List < String >$$

and

$$List < Integer >$$

do not correspond to two different classes inside the JVM. Instead, both are represented by the same runtime class:

$$List$$

The JVM runtime does not maintain separate metadata, object layouts, or method implementations for different generic arguments.

2.2.1 Erasure of Generic Type Parameters

The Java compiler transforms parameterized types by replacing type variables with their erasures. According to the Java Language Specification, the erasure of an unbounded type variable is `Object`, while a bounded type variable is replaced by the erasure of its first bound.

Consider the generic declaration:

```
class Box<T> {
    private T value;

    T get() {
        return value;
    }
}
```

After erasure, the runtime representation is equivalent to:

```
class Box {
    private Object value;

    Object get() {
        return value;
    }
}
```

The JVM therefore executes a representation where the type variable T has been replaced by its erased type. The runtime system has no knowledge that the original source-level declaration used a generic parameter.

Formally, the transformation can be expressed as:

$$Box < T > \rightarrow Box < Object >$$

for an unbounded type variable.

Consequently, the runtime identity of an object is determined only by the erased class:

$$RuntimeType(Box < String >) = RuntimeType(Box < Integer >) = Box$$

The generic parameter is not part of the runtime type identity.

2.2.2 Object Layout and Storage Representation

Because Java Generics do not create specialized runtime classes, generic objects use the same memory representation as ordinary JVM objects.

A generic reference field:

$$T\ value$$

is represented after erasure as:

$$Object\ value$$

The JVM stores a reference to an object rather than a value specialized for the concrete generic type.

For example, an instance of:

$$ArrayList < String >$$

and an instance of:

$$ArrayList < Integer >$$

have identical internal structural representations. The JVM does not allocate a specialized array of strings or integers for these collections. The generic container stores references using the standard JVM reference representation.

This differs from a reified implementation, such as C++ template monomorphization, where each concrete instantiation may generate a distinct object layout and specialized machine code. In Java, the same erased representation is reused for all parameterizations.

2.2.3 Runtime Type Information

Java runtime type information is based on classes and interfaces, not on generic parameters.

The JVM can determine:

$$object instanceof ArrayList$$

because the class identity is preserved.

However, it cannot determine:

$$object instanceof ArrayList < String >$$

because the generic argument has been erased.

Therefore, generic type information has a limited runtime presence. The class file may contain a `Signature` attribute preserving generic declarations for tools, compilers, and reflection APIs, but this metadata does not participate in JVM execution, object layout, or dynamic dispatch.

The runtime system effectively observes:

$$ArrayList$$

rather than:

$$ArrayList < String >$$

2.2.4 Inserted Runtime Casts

Since generic parameters are removed before execution, the compiler inserts runtime casts whenever a value retrieved from a generic structure must be treated as a more specific type.

Example:

```
String s = list.get(0);
```

The erased execution model becomes equivalent to:

```
Object tmp = list.get(0);
String s = (String) tmp;
```

At the bytecode level, the JVM executes a `checkcast` instruction.

The runtime sequence is therefore:

retrieve erased reference → check actual object type → use typed reference

The runtime verification concerns the concrete class of the object, not the generic argument. The JVM checks whether the object is a `String`, but it does not verify the existence of an original `List<String>` instantiation.

2.2.5 Bridge Methods

Type erasure can modify method signatures and break overriding relationships at the JVM level. To preserve polymorphism, the Java compiler generates synthetic *bridge methods*.

Example:

```
class Node<T> {
    T get();
}

class StringNode extends Node<String> {
    String get();
}
```

Before erasure, the overriding relationship is:

$$Node < T > .get() : T$$

overridden by:

$$StringNode.get() : String$$

After erasure, the parent method becomes:

$$get() : Object$$

while the child method remains:

$$get() : String$$

Since the JVM method descriptors differ, the compiler generates a bridge:

```
Object get() {
    return get();
}
```

The bridge method adapts the specialized method to the erased representation, allowing normal JVM method dispatch to operate correctly.

Bridge methods are therefore a direct runtime consequence of the decision to implement generics through erasure rather than through specialized generated types.

2.2.6 Primitive Types and Generic Representation

Java Generics are restricted to reference types because erased generic parameters are represented using reference-compatible types.

A primitive instantiation:

$$List < int >$$

is not permitted.

Instead, the corresponding wrapper type must be used:

$$List < Integer >$$

The runtime representation therefore becomes:

$$int \rightarrow Integer \rightarrow Object$$

The primitive value is converted into an object representation through boxing, and retrieved values require unboxing.

This behavior follows directly from the erased representation: since every generic parameter is mapped to a reference type, the JVM cannot create a specialized generic representation containing primitive values.

2.2.7 Comparison with C++ Templates

C++ Templates are based on a different runtime representation strategy: *monomorphization*. Unlike Java Generics, C++ compilers generate a separate representation for each concrete template instantiation.

This difference explains why Java uses one erased runtime representation:

$$List < T > \rightarrow List < Object >$$

while C++ can generate independent specialized representations:

$$Vector < int >,\ Vector < double >, \ldots$$

The Java approach avoids multiple generated implementations but sacrifices runtime visibility of generic parameters and requires mechanisms such as casts, boxing, and bridge methods. C++ preserves concrete type information in the generated executable by creating specialized versions, but this may increase generated code size.

The central distinction is therefore:

Java Generics preserve abstraction through erasure

whereas:

C++ Templates preserve abstraction through specialization

For Java, the generic type parameter is a compile-time concept. For C++, template instantiation becomes part of the generated runtime representation.

Chapter 3

C# Generics

3.1 Instantiation Time

C# generics adopt a runtime-oriented instantiation model that differs fundamentally from both the compile-time monomorphization strategy of C++ templates and the type erasure strategy of Java generics. The defining characteristic of the C# approach is that generic type definitions are compiled before execution, while the generation of their concrete executable representation is delayed until runtime by the Common Language Runtime (CLR) and its Just-In-Time (JIT) compiler.

In C#, generic declarations are not immediately expanded into specialized versions during compilation. Instead, the C# compiler translates generic classes and methods into Common Intermediate Language (CIL), preserving generic parameters as metadata entities. The resulting assembly contains a generic type definition together with information about its type parameters and constraints, but it does not necessarily contain native code for each possible generic instantiation.

For example, the generic definition:

```
class Container<T>
{
    public T value;
}
```

is compiled once into a generic CLI type definition. The compiler does not generate separate implementations such as:

```
Container<int>
Container<double>
Container<string>
```

at compilation time. The concrete instantiation is postponed until execution.

3.1.1 CLR and JIT Generic Instantiation Mechanism

The instantiation process begins when the CLR loads an assembly containing a generic definition. At this stage, the runtime knows the existence of the generic type but does not necessarily generate machine code for any concrete specialization.

A concrete generic instantiation is created when program execution requires a specific constructed type. For example, when the runtime encounters:

```
Container<int> c;
```

the CLR resolves the generic definition `Container<T>` together with the type argument `int`. This produces a runtime representation of the constructed generic type `Container<int>`.

Only when executable code for this instantiation is required does the JIT compiler translate the corresponding CIL into native machine instructions. Therefore, the timeline of C# generic instantiation can be summarized as:

1. The C# compiler generates a generic CIL definition.
2. The assembly is loaded by the CLR.
3. A concrete generic instantiation is requested during execution.
4. The CLR creates the constructed generic type.
5. The JIT compiler generates native code when required.

Consequently, C# generics are instantiated at runtime rather than during source compilation.

The CLR does not necessarily create a completely independent native implementation for every generic instantiation. Instead, the JIT compiler applies different strategies depending on the category of the type argument.

For reference types, the CLR can usually share the same generated native code between different instantiations because all reference types have compatible managed reference representations. Therefore, instantiations such as:

```
List<string>
List<object>
List<MyClass>
```

may reuse a single JIT-generated implementation.

For value types, the situation is different. Value types have different memory layouts and sizes, meaning that a single shared implementation cannot always be used. Therefore, the CLR JIT compiler generates specialized native code for different value-type instantiations:

```
List<int>
List<double>
```

represent separate JIT specialization events.

Thus, the C# instantiation model is hybrid: runtime instantiation occurs in both cases, but reference types generally use runtime code sharing, whereas value types trigger runtime specialization.

3.1.2 Comparison with C++ Templates: Runtime Instantiation vs Compile-Time Instantiation

The C# runtime instantiation model differs from the C++ template model primarily because the moment at which specialization occurs is different.

C++ templates use compile-time instantiation, commonly described as monomorphization. When the compiler encounters:

```
Vector<int>
Vector<double>
```

it generates separate concrete implementations during compilation. The executable produced by the compiler already contains the specialized versions before execution.

The timeline is therefore:

Template Definition → Compile-Time Instantiation → Native Code Generation → Execution

In contrast, the C# timeline is:

Generic Definition → CIL Generation → Runtime Instantiation → JIT Compilation → Execution

The essential difference is that C++ performs specialization ahead of execution, whereas C# delays specialization until the runtime environment determines that a specific generic instantiation is required.

3.1.3 Comparison with Java Generics: Runtime Instantiation vs Type Erasure

Java generics follow a fundamentally different model based on type erasure. During Java compilation, generic parameters are removed and replaced by their erasures. Consequently, the Java Virtual Machine does not instantiate generic types at runtime.

For example:

```
List<String>
List<Integer>
```

are compiled into the same erased representation:

```
List
```

The JVM executes a single non-specialized implementation, and no runtime generic instantiation event occurs.

The timeline of Java generics is therefore:

Generic Source Code → Compile-Time Erasure → Bytecode Generation → Execution

This differs from C#, where the generic definition survives compilation and concrete instantiation is performed during execution.

Therefore, the three models can be summarized as:

The defining property of C# generics is therefore not merely the preservation of generic information, but the placement of the instantiation event: generic types are materialized by the runtime system when execution requires them, with the JIT compiler acting as the specialization mechanism.

Language	Instantiation Time	Specialization Phase
C++	Compile time	Compiler monomorphization
C#	Runtime	CLR/JIT instantiation
Java	Compile time	Type erasure, no specialization

3.2 Runtime Representation

C# generics provide a runtime implementation of parametric polymorphism based on *reification*. In a reified generic system, information about generic type parameters is preserved after compilation and remains available to the execution environment. This design differs fundamentally from the two alternative approaches adopted by Java and C++.

From a theoretical perspective, parametric polymorphism allows a program component to operate independently from the concrete types on which it is instantiated. A generic definition can be represented as:

$$G\langle T\rangle$$

where T is a type variable that can later be replaced by a concrete type argument. However, the concept of parametric polymorphism does not define how generic abstractions are represented at runtime. The implementation requires decisions about metadata representation, code generation, object layout, and execution mechanisms.

The CLR implements generics through a hybrid model that combines runtime reification with selective specialization. Generic definitions are preserved inside compiled assemblies, while the final executable representation is generated dynamically by the Just-In-Time (JIT) compiler. Therefore, unlike C++ templates, generic code is not fully expanded during compilation, and unlike Java generics, generic parameters are not removed before execution.

Conceptually, the C# compilation pipeline can be represented as:

C# Source → CIL + Metadata → CLR Generic Instantiation → JIT Compilation → Native Code

This architecture allows the runtime to preserve generic type identity while deciding dynamically whether a particular instantiation requires a specialized implementation or whether an existing implementation can be shared.

3.2.1 CLR Execution Engine and CIL Metadata Representation

The first stage of the CLR generic implementation occurs during compilation. The C# compiler translates source code into *Common Intermediate Language* (CIL) and generates metadata according to the Common Language Infrastructure (CLI) specification defined by ECMA-335.

Unlike Java bytecode after type erasure, the generated assembly preserves explicit information about generic declarations. A generic type is represented as an open generic definition containing formal type parameters.

For example:

`List<T>`

is stored as a generic type definition rather than as a concrete implementation.

The CLR metadata system contains dedicated structures for representing generic relationships. The most relevant metadata tables include:

- `GenericParam`, which stores information about generic parameters;
- `GenericParamConstraint`, which stores constraints associated with generic parameters;
- `TypeSpec`, which represents constructed types;
- `MethodSpec`, which represents constructed generic methods.

These structures allow the CLR loader and execution engine to reconstruct the relationship between a generic definition and its concrete arguments.

For example, the runtime can represent:

`List<T>` $\rightarrow$ `List<int>`

as an explicit construction relationship.

This differs from Java's type-erasure model, where:

`List<String>`

and:

`List<Integer>`

are both transformed into the same runtime class representation:

`List`

The JVM therefore loses generic type identity during execution, while the CLR maintains it through runtime metadata.

3.2.2 Runtime Instantiation and JIT Compilation

A fundamental characteristic of CLR generics is that instantiation occurs at runtime rather than during the initial compilation phase. The compiler emits generic definitions, while the CLR creates concrete representations when a closed constructed type is required.

A generic definition:

$$G\langle T\rangle$$

becomes a runtime constructed type:

$$G\langle U\rangle$$

where U is a concrete type argument.
For example:

```
Dictionary<TKey,TValue>
```

may generate runtime instantiations such as:

```
Dictionary<int,string>
```

or:

```
Dictionary<Guid,Object>
```

The CLR execution engine resolves these constructed types and provides the necessary information to the JIT compiler. The JIT compiler then generates native machine code according to the characteristics of the supplied type arguments.
This differs from C++ template instantiation, where:

$$Compiler(G\langle int\rangle) = MachineCode_{int}$$

occurs during compilation. In C++, the runtime only observes the final native artifacts. In contrast, the CLR retains the generic abstraction and performs the instantiation process as part of runtime execution.

3.2.3 Value-Type Specialization

When a generic type is instantiated with a value type, the CLR generally produces a specialized native implementation for that concrete type. Value types such as `int`, `double`, or user-defined structures have their own memory layouts and representation requirements. Consequently, sharing a single machine-code implementation between unrelated value types would require additional abstraction mechanisms.
For example:

```
List<int>
```

and:

```
List<double>
```

require different representations of their stored elements. Therefore, the JIT compiler generates specialized code:

$$JIT(\texttt{List<int>}) = NativeCode_{int}$$

and:

$$JIT(\texttt{List<double>}) = NativeCode_{double}$$

The generated implementations contain concrete type information and operate directly on the corresponding value representation.

This behavior resembles C++ template monomorphization at the level of generated machine code. However, the architectural difference is that C++ performs this transformation statically during compilation, while the CLR performs specialization inside the runtime environment.

3.2.4 Reference-Type Code Sharing

Generic instantiations based on reference types follow a different execution strategy. Because all reference types share a common object-reference representation inside the CLR, the runtime can reuse the same native implementation for multiple generic instantiations.

For example:

```
List<string>
```

and:

```
List<object>
```

may share the same generated machine code.

Conceptually, instead of producing:

$$JIT(\texttt{List<string>}) = NativeCode_{string}$$

and:

$$JIT(\texttt{List<object>}) = NativeCode_{object}$$

the CLR can generate a shared implementation:

$$NativeCode_{reference}$$

where runtime-specific information is supplied through additional execution-engine structures.

This mechanism provides a balance between specialization and code reuse. Unlike C++ templates, the CLR does not generate an independent implementation for every reference-type instantiation. Unlike Java erasure, however, the runtime still knows the exact generic type arguments and can preserve type safety.

3.2.5 Runtime Dictionaries and Method Tables

To support shared generic implementations, the CLR uses runtime data structures that provide information about concrete generic arguments. These structures are commonly described as generic dictionaries or runtime generic contexts.

When shared machine code executes, operations requiring type-specific information can access these runtime structures to resolve:

- type descriptors;
- method addresses;

- field layouts;
- interface implementations.

This mechanism is conceptually related to dictionary passing techniques used in implementations of parametric polymorphism. The generated code remains generic, while additional runtime information supplies the missing concrete type details.

The execution model therefore differs from C++ virtual tables and RTTI. C++ templates do not require runtime generic dictionaries because all template parameters have already been resolved before execution. Conversely, Java erasure does not maintain such information because generic parameters are removed before bytecode execution.

3.2.6 Runtime Type Identity and Reflection

A defining property of CLR reification is that constructed generic types maintain their runtime identity. The CLR considers different generic instantiations as distinct runtime types.

Therefore:

$$RuntimeType(\texttt{List<int>}) \neq RuntimeType(\texttt{List<string>})$$

The reflection system can retrieve generic information through runtime type objects, allowing programs to inspect:

- generic type definitions;
- concrete type arguments;
- generic constraints.

This capability is fundamentally different from Java, where:

$$RuntimeType(\texttt{List<String>}) = RuntimeType(\texttt{List<Integer>}) = \texttt{List}$$

after erasure.

C++ occupies a different position. Concrete template specializations may participate in RTTI if they correspond to runtime types, but the original template definition is not preserved as a runtime entity. The executable contains information about concrete types rather than about the generic abstraction that produced them.

3.2.7 Architectural Comparison

The runtime representation strategies can be summarized as follows:

Feature	C# Generics	Java Generics	C++ Templates
Generic model	Reification	Type Erasure	Monomorphization
Metadata presence	Preserved in CLR	Removed from JVM execution	Not available
Instantiation time	Runtime/JIT	Compile-time only	Compile-time
Code generation	Specialization and sharing	Single erased bytecode	Native specialization
Runtime identity	Generic arguments preserved	Generic arguments removed	Concrete types only
Generic runtime object	Yes	No	No

Table 3.1: Comparison of generic runtime representation models.